EXPERIMENT - 11

AIM :

To implement the Map Coloring problem using Constraint Satisfaction Problem (CSP).

ALGORITHM:

1. Start the program.
2. Define the regions, neighboring regions, and available colors.
3. Check whether neighboring regions have the same color.
4. Assign a valid color to each region using backtracking.
5. Display the color assigned to each region.
6. Stop the program.

PROGRAM :

untitled (3.14.8)

File Edit Format Run Options Window Help

```
regions = ['A', 'B', 'C', 'D']
colors = ['Red', 'Green', 'Blue']

neighbors = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

assignment = {}

def solve(index):
    if index == len(regions):
        return True

    region = regions[index]

    for color in colors:
        if all(assignment.get(n) != color
               for n in neighbors[region]):
            assignment[region] = color

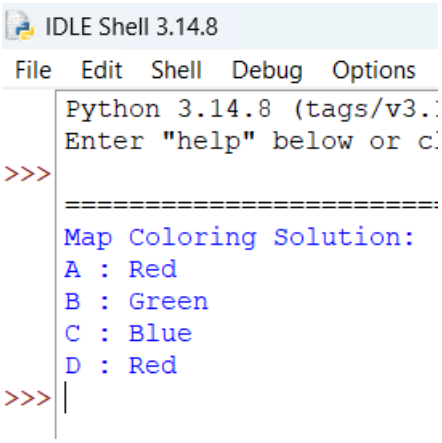
            if solve(index + 1):
                return True

            del assignment[region]

    return False

if solve(0):
    print("Map Coloring Solution:")
    for region, color in assignment.items():
        print(region, ":", color)
else:
    print("No solution exists")
```

OUTPUT:



RESULT:

Thus, the Map Coloring problem was implemented successfully using CSP and backtracking.